CHAPTER 05

SQL & PostgreSQL

Data Types · Joins · Window Fns · CTEs · JSONB · Partitioning

Chapter Overview

Chapter 05 covers SQL and PostgreSQL as used in ShopVerse: data types, DDL, DML, joins, aggregation, window functions, CTEs, indexes, query planning, partitioning, and PostgreSQL-specific features including jsonb, full-text search, and advisory locks.

Sub-Chapter	Topic	Key Concept
05-01	PostgreSQL Data Types	NUMERIC, TIMESTAMPTZ, JSONB, TEXT, BOOLEAN, SERIAL vs IDEN
05-02	DDL – Tables, Constraints, Indexes	CREATE TABLE, CHECK, FK, UNIQUE, partial indexes
05-03	DML – CRUD & Upsert	SELECT, INSERT, UPDATE, DELETE, INSERT … ON CONFLICT
05-04	Joins	INNER, LEFT, RIGHT, FULL, CROSS, LATERAL join with examples
05-05	Aggregation & GROUP BY	COUNT, SUM, AVG, MIN, MAX, HAVING, ROLLUP, GROUPING SETS
05-06	Window Functions	RANK, DENSE_RANK, ROW_NUMBER, LEAD, LAG, SUM OVER, PART
05-07	CTEs & Recursive Queries	WITH, WITH RECURSIVE, chained CTEs, DELETE with CTE
05-08	Query Planning & EXPLAIN	EXPLAIN ANALYZE, seq scan vs index scan, cost model
05-09	Partitioning	RANGE, LIST, HASH partitioning; partition pruning
05-10	PostgreSQL-Specific Features	JSONB operators, full-text search, advisory locks, LISTEN/NOTIFY
05-11	Transactions & Isolation in SQL	BEGIN, SAVEPOINT, isolation levels, NOWAIT
05-12	Stored Functions & Triggers	PL/pgSQL functions, triggers for auto-updated_at

05-01 PostgreSQL Data Types

Type	Storage	Range / Notes	ShopVerse Column
SMALLINT	2 B	-32 768 to 32 767	Never for IDs or amounts
INTEGER	4 B	~-2.1B to ~2.1B	page, sort_order (not IDs – overflow risk)
BIGINT	8 B	~±9.2×10¹■	ALL entity IDs, foreign keys
NUMERIC(p,s)	variable	Exact decimal; p=precision, s=scale	NUMERIC(19,4) for all monetary amounts
REAL / DOUBLE PRECISION	4/8 B	IEEE-754 float	Never for money; OK for geo/stats
BOOLEAN	1 B	true/false	is_active, is_featured, is_deleted
VARCHAR(n)	4B + n	At most n chars	status VARCHAR(30), type VARCHAR(20)
TEXT	4B + chars	Unlimited	description, body, review_text
TIMESTAMP	8 B	No timezone – stores as-is	Avoid – use TIMESTAMPTZ
TIMESTAMPTZ	8 B	UTC stored; displayed in session tz	created_at, updated_at, shipped_at
DATE	4 B	Calendar date only	report_date, expiry_date
INTERVAL	16 B	Time interval	TTL values, subscription periods
UUID	16 B	128-bit random identifier	external_ref UUID DEFAULT gen_random_uuid()
JSONB	variable (binary indexed)	JSON stored as binary; supports GIN index	product metadata, flexible attributes
BYTEA	variable	Raw bytes	Encrypted fields, file chunks (rare)
ARRAY	variable	Any type[]	tags TEXT[], phone_numbers TEXT[]

SERIAL vs IDENTITY vs Sequence

Approach	Syntax	Notes	ShopVerse Verdict
SERIAL (legacy)	col SERIAL	Creates implicit sequence; owned by column	Avoid – more verbose
BIGSERIAL	col BIGSERIAL	Same as SERIAL but BIGINT	Avoid – non-standard
IDENTITY (standard)	col BIGINT GENERATED BY DEFAULT AS IDENTITY	SQL standard; sequence created automatically	Use for all PKs ■
Manual SEQUENCE	CREATE SEQUENCE; DEFAULT nextval('seq')	Explicit control over allocation size	Use when Hibernate needs SEQUENCE generator

Complete Table Definition Example

```
CREATE TABLE products (
id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
name VARCHAR(255) NOT NULL,
description TEXT,
price NUMERIC(19,4) NOT NULL,
category_id BIGINT NOT NULL REFERENCES categories(id) ON DELETE RESTRICT,
stock INTEGER NOT NULL DEFAULT 0,
is_active BOOLEAN NOT NULL DEFAULT true,
attributes JSONB, -- flexible product attributes
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
version INTEGER NOT NULL DEFAULT 0,
-- constraints
CONSTRAINT chk_price_positive CHECK (price >= 0),
CONSTRAINT chk_stock_non_neg CHECK (stock >= 0)
);
-- Indexes
CREATE INDEX idx_products_category ON products(category_id);
CREATE INDEX idx_products_active_price ON products(category_id, is_active, price)
WHERE is_active = true; -- partial index – only active products
CREATE INDEX idx_products_fts ON products
USING GIN (to_tsvector('english', name || ' ' || COALESCE(description, '')));
```

Constraint Types Reference

Constraint	DDL Syntax	Violation Exception	ShopVerse Usage
PRIMARY KEY	CONSTRAINT pk PRIMARY KEY (id)	ERROR 23505	All tables
FOREIGN KEY	REFERENCES other(id) ON DELETE RESTRICT	ERROR 23503	order_items → orders, products
UNIQUE	CONSTRAINT uq UNIQUE (email)	ERROR 23505	customers.email, products.sku
CHECK	CONSTRAINT chk CHECK (price >= 0)	ERROR 23514	price, stock, quantity
NOT NULL	col TYPE NOT NULL	ERROR 23502	All required fields
EXCLUSION	CONSTRAINT exc EXCLUDE USING GIST (...)	ERROR 23P01	Date range overlaps (schedules)

05-03 DML – CRUD & Upsert

```
-- SELECT with filtering, sorting, pagination
SELECT p.id, p.name, p.price, c.name AS category
FROM products p
JOIN categories c ON c.id = p.category_id
WHERE p.is_active = true
AND p.category_id = $1
AND p.price BETWEEN $2 AND $3
ORDER BY p.price ASC, p.name ASC
LIMIT $4 OFFSET $5;
```

```
-- INSERT with returning (get generated ID)
INSERT INTO orders (customer_id, status, total_amount, created_at)
VALUES ($1, 'PENDING', $2, NOW())
RETURNING id, created_at;
```

```
-- UPDATE with optimistic lock check
UPDATE orders
SET status = $2, updated_at = NOW(), version = version + 1
WHERE id = $1 AND version = $3;
-- If 0 rows affected → OptimisticLockException
```

```
-- DELETE with condition
DELETE FROM cart_items WHERE cart_id = $1 AND product_id = $2;
```

```
-- UPSERT (INSERT … ON CONFLICT DO UPDATE)
INSERT INTO inventory (product_id, warehouse_id, stock_count)
VALUES ($1, $2, $3)
ON CONFLICT (product_id, warehouse_id) DO UPDATE
SET stock_count = EXCLUDED.stock_count,
updated_at = NOW()
RETURNING stock_count;
```

```
-- Bulk INSERT from VALUES
INSERT INTO order_items (order_id, product_id, qty, unit_price)
SELECT $1, p.id, v.qty, p.price
FROM products p
JOIN (VALUES ($2::bigint,$3::int), ($4,$5)) AS v(product_id, qty)
ON p.id = v.product_id;
```

Join Types Reference

Join Type	Returns	SQL Syntax	ShopVerse Example
INNER JOIN	Only matching rows in both tables	JOIN / INNER JOIN	orders JOIN customers ON …
LEFT JOIN	All left + matching right (NULL if no match)	LEFT JOIN	orders LEFT JOIN coupons ON …
RIGHT JOIN	All right + matching left	RIGHT JOIN	Rare; rewrite as LEFT JOIN
FULL OUTER JOIN	All rows from both; NULL when no match	FULL OUTER JOIN	Reconciliation queries
CROSS JOIN	Cartesian product (every combo)	CROSS JOIN	Generate test data; small dimension tables
SELF JOIN	Table joined to itself	t1 JOIN t1 t2 ON …	Category parent→child hierarchy
LATERAL JOIN	Subquery can reference outer row	JOIN LATERAL	Top-N per group; correlated subqueries

```
-- LEFT JOIN: orders with optional coupon info
SELECT o.id, o.total_amount, c.code AS coupon_used
FROM orders o
LEFT JOIN coupons c ON c.id = o.coupon_id
WHERE o.customer_id = $1
ORDER BY o.created_at DESC;
```

```
-- LATERAL JOIN: top 3 products per category
SELECT cat.name, top.name AS top_product, top.price
FROM categories cat
JOIN LATERAL (
SELECT p.name, p.price
FROM products p
WHERE p.category_id = cat.id AND p.is_active = true
ORDER BY p.sales_count DESC
LIMIT 3
) AS top ON true;
```

05-05 Aggregation & GROUP BY

Aggregate Functions Reference

Function	Returns	Ignores NULL?	ShopVerse Use
COUNT(*)	Count of all rows	No	Total orders, total products
COUNT(col)	Count of non-NULL values	Yes	Count orders with coupon_id
COUNT(DISTINCT col)	Count of unique values	Yes	Count distinct customers who ordered
SUM(col)	Total of numeric column	Yes	Total revenue, total items sold
AVG(col)	Average	Yes	Average order value, average rating
MIN(col) / MAX(col)	Minimum / Maximum	Yes	Earliest order, highest sale price
BOOL_AND / BOOL_OR	All/any true	Yes	All items in stock, any discount applied
STRING_AGG(col,sep)	Concatenate strings	Yes	Comma-separated tag list per product
ARRAY_AGG(col)	Aggregate into array	Yes	Array of order IDs per customer

```
-- Revenue by category with HAVING filter
SELECT c.name AS category,
COUNT(DISTINCT o.id) AS order_count,
SUM(oi.qty * oi.unit_price) AS revenue,
AVG(oi.unit_price) AS avg_price
FROM order_items oi
JOIN products p ON p.id = oi.product_id
JOIN categories c ON c.id = p.category_id
JOIN orders o ON o.id = oi.order_id
WHERE o.status = 'DELIVERED'
GROUP BY c.id, c.name
HAVING SUM(oi.qty * oi.unit_price) > 10000
ORDER BY revenue DESC;
```

```
-- ROLLUP – subtotals and grand total
SELECT category_id, status, SUM(total_amount)
FROM orders
GROUP BY ROLLUP(category_id, status);
-- Generates: (category, status), (category, NULL=subtotal), (NULL,NULL=grand total)
```

Window Function Reference

Function	Purpose	Resets on Partition?
ROW_NUMBER()	Unique row number within partition	Yes
RANK()	Rank with gaps (1,1,3 for ties)	Yes
DENSE_RANK()	Rank without gaps (1,1,2 for ties)	Yes
LEAD(col, n)	Value from n rows ahead in partition	Yes
LAG(col, n)	Value from n rows behind in partition	Yes
FIRST_VALUE(col)	First value in partition window	Yes
LAST_VALUE(col)	Last value in partition window	Yes
SUM(col) OVER(…)	Running or partitioned total	Yes (within partition)
AVG(col) OVER(…)	Moving average	Yes
NTILE(n)	Divide into n buckets (quartiles, deciles)	Yes

```
-- Top 3 products per category by sales_count
SELECT *
FROM (
SELECT p.id, p.name, p.category_id,
RANK() OVER (PARTITION BY p.category_id ORDER BY p.sales_count DESC) AS rnk
FROM products p
WHERE p.is_active = true
) ranked
WHERE rnk <= 3;
```

```
-- Month-over-month revenue comparison using LAG
SELECT month,
revenue,
LAG(revenue) OVER (ORDER BY month) AS prev_month_rev,
revenue - LAG(revenue) OVER (ORDER BY month) AS mom_change
FROM (
SELECT DATE_TRUNC('month', created_at) AS month,
SUM(total_amount) AS revenue
FROM orders WHERE status = 'DELIVERED'
GROUP BY 1
) monthly
ORDER BY month;
```

```
-- Running total of daily orders
SELECT order_date, daily_count,
SUM(daily_count) OVER (ORDER BY order_date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
AS running_total
FROM (SELECT DATE(created_at) AS order_date, COUNT(*) AS daily_count
FROM orders GROUP BY 1) daily;
```

05-07 CTEs & Recursive Queries

```
-- Simple CTE – readable multi-step query
WITH active_products AS (
SELECT id, name, price, category_id
FROM products WHERE is_active = true
),
category_stats AS (
SELECT category_id, AVG(price) AS avg_price, COUNT(*) AS cnt
FROM active_products
GROUP BY category_id
)
SELECT p.name, p.price,
cs.avg_price,
p.price - cs.avg_price AS diff_from_avg
FROM active_products p
JOIN category_stats cs ON cs.category_id = p.category_id
ORDER BY ABS(p.price - cs.avg_price) DESC
LIMIT 20;
```

```
-- RECURSIVE CTE – category hierarchy
WITH RECURSIVE category_tree AS (
-- anchor: root categories
SELECT id, name, parent_id, 0 AS depth, name::TEXT AS path
FROM categories WHERE parent_id IS NULL
UNION ALL
-- recursive: children
SELECT c.id, c.name, c.parent_id, ct.depth+1,
ct.path || ' > ' || c.name
FROM categories c
JOIN category_tree ct ON c.parent_id = ct.id
)
SELECT id, name, depth, path
FROM category_tree
ORDER BY path;
```

```
-- CTE with DELETE – clean expired sessions
WITH expired AS (
DELETE FROM sessions WHERE expires_at < NOW()
RETURNING id, user_id
)
INSERT INTO session_audit(session_id, user_id, expired_at)
SELECT id, user_id, NOW() FROM expired;
```

EXPLAIN ANALYZE Output Anatomy

```
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT * FROM orders WHERE customer_id = 42 ORDER BY created_at DESC LIMIT 10;
-- Sample output:
Limit (cost=0.43..8.45 rows=10 width=120) (actual time=0.123..0.145 rows=10 loops=1)
-> Index Scan using idx_orders_customer_id on orders
(cost=0.43..82.1 rows=100 width=120) (actual time=0.120..0.139 rows=10 loops=1)
Index Cond: (customer_id = 42)
Buffers: shared hit=14
```

EXPLAIN Term	Meaning	Action If Bad
cost=X..Y	X=startup cost, Y=total cost (planner estimate)	Stale statistics → ANALYZE table
actual time=A..B	A=first row time, B=total time (ms)	B very high → index missing or bad query plan
rows=N (estimate)	Planner row estimate	Large diff vs actual → run ANALYZE to update stats
Seq Scan	Full table scan	Add index on filter/join column
Index Scan	B-tree lookup + row fetch	Good; normal for low-cardinality filter
Index Only Scan	Data from index; no heap fetch	Best case; covering index used
Bitmap Heap Scan	Index → bitmap → heap	Normal for medium result sets
Hash Join	Build hash table; probe it	Good for large equi-joins
Nested Loop	Outer row × inner scan	Bad for large outer; good for small outer
Buffers: hit vs read	shared hit=cache hit; read=disk read	High read count → warm OS cache or add RAM

Partition Strategies

Strategy	Key	Partition By	Pruning Trigger	ShopVerse Use
RANGE	Date, Number	CREATE TABLE … PARTITION BY RANGE(created_at)	WHERE created_at BETWEEN	orders table – monthly partitions
LIST	Enum, Status	PARTITION BY LIST(status)	WHERE status =	order_items by status
HASH	ID	PARTITION BY HASH(id)	WHERE id = (if hash known)	Even distribution across shards

```
-- Range partitioning: orders by month
CREATE TABLE orders (
id BIGINT NOT NULL,
created_at TIMESTAMPTZ NOT NULL,
total_amount NUMERIC(19,4),
-- ... other columns
PRIMARY KEY (id, created_at) -- partition key must be in PK
) PARTITION BY RANGE (created_at);
CREATE TABLE orders_2025_01 PARTITION OF orders
FOR VALUES FROM ('2025-01-01') TO ('2025-02-01');
CREATE TABLE orders_2025_02 PARTITION OF orders
FOR VALUES FROM ('2025-02-01') TO ('2025-03-01');
-- Query automatically prunes to relevant partition(s)
SELECT COUNT(*) FROM orders WHERE created_at >= '2025-01-01' AND created_at < '2025-02-01';
-- EXPLAIN shows only orders_2025_01 is scanned
```

JSONB Operators

Operator	Meaning	Example
->	Get JSON field (returns jsonb)	attributes -> 'color' → "red"
->>	Get JSON field as text	attributes ->> 'color' → red
#>	Get nested field (path array)	attributes #> '{dimensions,width}'
#>>	Get nested field as text	attributes #>> '{dimensions,width}'
@>	Contains (left contains right)	attributes @> '{"color":"red"}'
<@	Contained by	'{"color":"red"}' <@ attributes
?	Key exists	attributes ? 'warranty'
?|	Any key exists	attributes ?| ARRAY['color','size']
jsonb_set	Update field	jsonb_set(attributes, '{color}', '"blue"')

```
-- Find products with warranty attribute
SELECT id, name, attributes ->> 'warranty' AS warranty
FROM products WHERE attributes ? 'warranty';
-- GIN index speeds up @> operator
CREATE INDEX idx_products_attributes ON products USING GIN (attributes);
```

Full-Text Search

```
-- GIN index on tsvector
CREATE INDEX idx_products_fts ON products
USING GIN (to_tsvector('english', name || ' ' || COALESCE(description, '')));
-- Search query
SELECT id, name, ts_rank(to_tsvector('english', name||description), query) AS rank
FROM products, to_tsquery('english', 'wireless & headphones') AS query
WHERE to_tsvector('english', name || ' ' || description) @@ query
ORDER BY rank DESC;
```

Advisory Locks

```
-- Application-level lock (not tied to a table row)
-- FlashSaleService: ensure only one process handles flash sale at a time
SELECT pg_try_advisory_lock(12345); -- returns true if acquired, false if already held
-- ... do flash sale processing ...
SELECT pg_advisory_unlock(12345); -- release when done
```

05-11 Transactions & Isolation in SQL

Transaction Control

```
-- Explicit transaction block
BEGIN;
UPDATE inventory SET stock = stock - $1 WHERE product_id = $2;
INSERT INTO order_items (order_id, product_id, qty) VALUES ($3, $2, $1);
UPDATE orders SET status = 'CONFIRMED' WHERE id = $3;
COMMIT;
-- On any error:
ROLLBACK;
```

```
-- SAVEPOINT for partial rollback
BEGIN;
INSERT INTO orders … ;
SAVEPOINT after_order;
INSERT INTO notifications … ; -- might fail
-- If notification fails:
ROLLBACK TO SAVEPOINT after_order; -- undo only notification insert
-- Order is still saved
COMMIT;
```

Lock Modes Reference

Lock Mode	SQL	Blocks	ShopVerse Use
SELECT FOR UPDATE	SELECT … FOR UPDATE	Other SELECT FOR UPDATE, UPDATE, DELETE	Lock inventory row before reserving for order
SELECT FOR SHARE	SELECT … FOR SHARE	FOR UPDATE only	Read a row, prevent concurrent updates
SKIP LOCKED	SELECT … FOR UPDATE SKIP LOCKED	Locked rows	Queue consumer – each worker picks different rows
NOWAIT	SELECT … FOR UPDATE NOWAIT	Returns error immediately if locked	Fail-fast alternative to waiting

```
-- SKIP LOCKED: concurrent order workers
SELECT id FROM orders WHERE status = 'PENDING'
ORDER BY created_at LIMIT 5
FOR UPDATE SKIP LOCKED;
-- Each worker gets different 5 rows; no waiting or duplicate processing
```

05-12 Stored Functions & Triggers

```
-- PL/pgSQL trigger: auto-update updated_at
CREATE OR REPLACE FUNCTION update_updated_at()
RETURNS TRIGGER AS $$
BEGIN
NEW.updated_at = NOW();
RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER trg_orders_updated_at
BEFORE UPDATE ON orders
FOR EACH ROW EXECUTE FUNCTION update_updated_at();
-- Function: calculate order total
CREATE OR REPLACE FUNCTION calculate_order_total(p_order_id BIGINT)
RETURNS NUMERIC AS $$
DECLARE
v_total NUMERIC(19,4);
BEGIN
SELECT SUM(qty * unit_price)
INTO v_total
FROM order_items
WHERE order_id = p_order_id;
RETURN COALESCE(v_total, 0);
END;
$$ LANGUAGE plpgsql;
-- Call the function
SELECT calculate_order_total(42);
```

BP Best Practices & Anti-Patterns

Do

- ✓ Use NUMERIC(19,4) for all monetary amounts – never FLOAT or DOUBLE.
- ✓ Use TIMESTAMPTZ for all timestamps – stores as UTC; handles timezone offsets.
- ✓ Add partial indexes (WHERE clause) for queries that filter on a subset of rows.
- ✓ Use INSERT … ON CONFLICT DO UPDATE (upsert) instead of DELETE+INSERT for idempotency.
- ✓ EXPLAIN ANALYZE before deploying any new query; check for Seq Scans on large tables.
- ✓ Use SKIP LOCKED for concurrent queue consumers – no blocking or duplicate processing.
- ✓ Use CTEs for multi-step queries – improves readability and testability.
- ✓ Always include partition key in WHERE clause to trigger partition pruning.
- ✓ Use advisory locks for distributed application-level locking (flash sale, batch jobs).

Anti-Patterns

- ■ SELECT * in production – fetches unused columns; prevents index-only scans.
- ■ FLOAT/DOUBLE for money – 0.1 + 0.2 ≠ 0.3 in IEEE-754.
- ■ SERIAL instead of GENERATED AS IDENTITY – non-standard; harder to sequence control.
- ■ Unindexed foreign keys – every FK update/delete triggers seq scan on child table.
- ■ N+1 in raw SQL – loop of queries in app code; use JOIN or CTE instead.
- ■ TRUNCATE inside a transaction on partitioned table – can break partition metadata.
- ■ Storing JSON as TEXT – loses JSONB operators and GIN indexing.
- ■ Using LIKE '%term%' without full-text index – always results in sequential scan.
- ■ Long-running transactions with locks – causes table bloat and blocks vacuum.

QR Quick Reference

Sub-Chapter	Rule	One-Liner
05-01 Types	NUMERIC(19,4)	Exact decimal for money; TIMESTAMPTZ for dates
05-02 DDL	Partial indexes	WHERE clause on index reduces size and improves speed
05-03 DML	ON CONFLICT DO UPDATE	Idempotent upsert – avoids race between SELECT+INSERT
05-04 Joins	LATERAL JOIN	Correlated subquery per outer row; top-N per group
05-05 Aggregation	ROLLUP	Subtotals and grand totals in one query
05-06 Window	RANK OVER (PARTITION BY)	Ranking within groups without collapsing rows
05-07 CTE	WITH RECURSIVE	Category/org hierarchy traversal
05-08 EXPLAIN	actual time vs cost	Large discrepancy = stale stats; run ANALYZE
05-09 Partition	PARTITION BY RANGE	Monthly partitions on orders; automatic pruning
05-10 JSONB	@> with GIN index	Fast JSON containment search on attributes
05-11 Locking	SKIP LOCKED	Concurrent workers take different queue rows
05-12 Triggers	update_updated_at()	Auto-set updated_at via BEFORE UPDATE trigger

Interview Quick-Check

Question	Concise Answer
Why TIMESTAMPTZ over TIMESTAMP?	TIMESTAMPTZ stores as UTC internally; converts to session timezone on read. TIMESTAMP has
Partial vs composite index?	Partial: WHERE clause restricts rows. Composite: multiple columns. Partial index is smaller and fas
RANK() vs DENSE_RANK()?	RANK: gaps after ties (1,1,3). DENSE_RANK: no gaps (1,1,2). Use DENSE_RANK for leaderboard
CTE materialization?	PostgreSQL 12+: CTEs not always materialized. Use MATERIALIZED/NOT MATERIALIZED hint w
SKIP LOCKED use case?	Job queue: each worker picks rows not locked by others. No polling loop needed; inherently concu